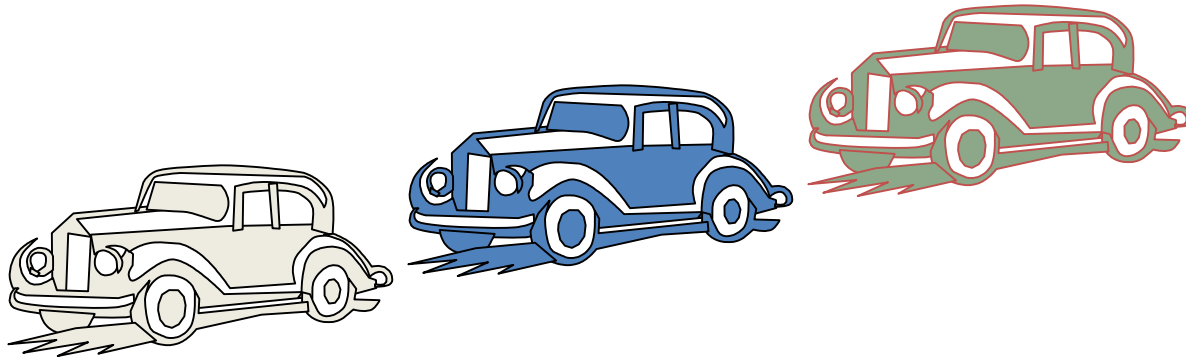


Queues

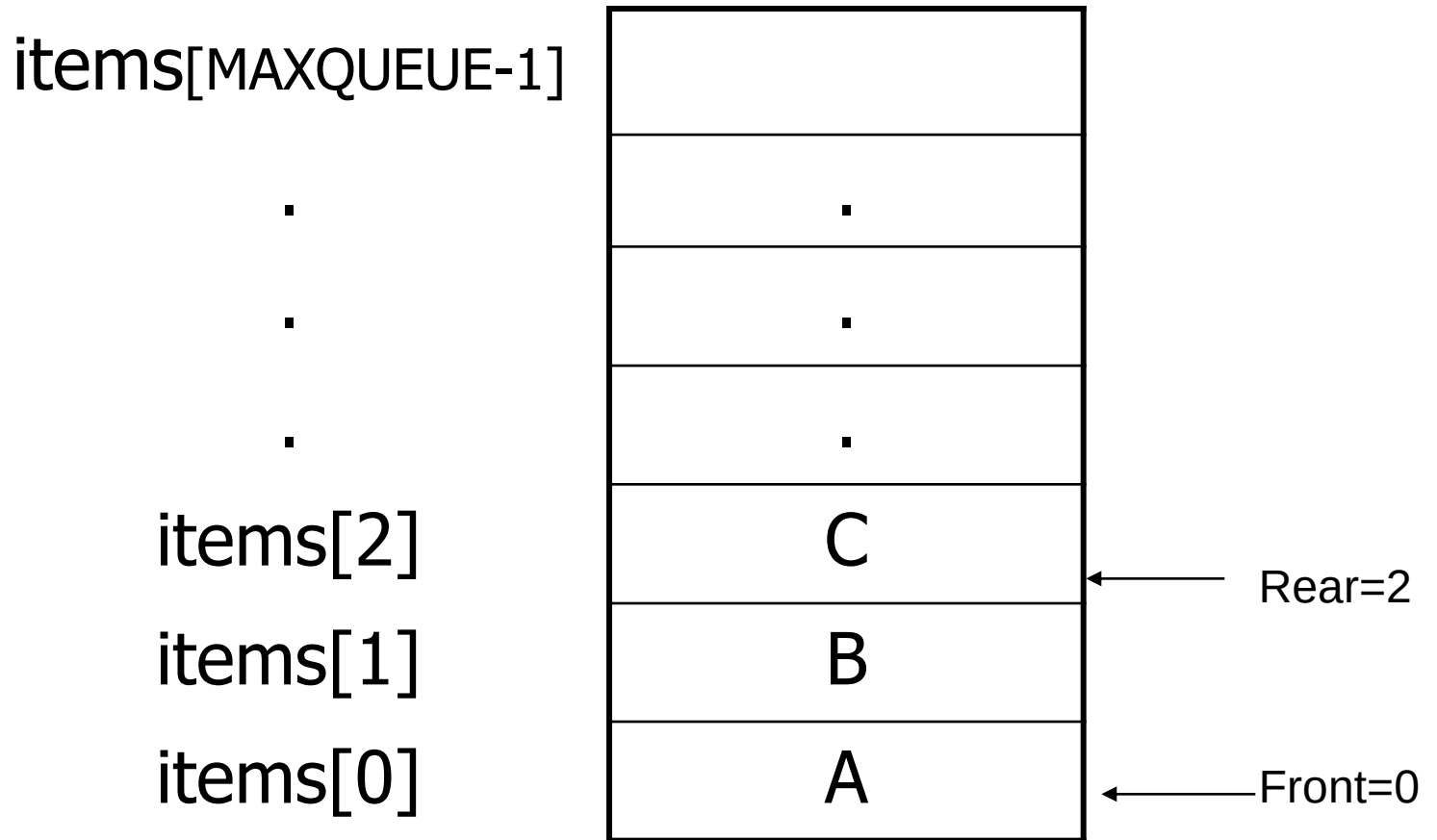


What is a Queue ?

- ◆ A **Queue** is an ordered collection of items from which items may be removed at one end (called the **front** of the queue) and into which items may be inserted at the other end (the **rear** of the queue).
- ◆ The first element inserted into the queue is the first element to be removed. For this reason a queue is sometimes called a **FIFO (First-In First-Out)** list as opposed to the stack, which is a **LIFO (Last-In First-Out)**



What is a Queue ?



The Queue ADT

- The **Queue** ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- Main queue operations:
 - **enqueue**(Object o): inserts an element o at the rear of the queue
 - **dequeue**(): removes and returns the element at the front of the queue
- Auxiliary queue operations:
 - **front**(): returns the element at the front without removing it
 - **size**(): returns the number of elements stored
 - **isEmpty**(): returns a Boolean indicating whether no elements are stored
- Exceptions
 - Attempting the execution of dequeue or front on an empty queue throws an **EmptyQueueException**

Applications of Queues

- Direct applications
 - Waiting lists, bureaucracy
 - Access to shared resources (e.g., printer)
 - Multiprogramming
- Indirect applications
 - Auxiliary data structure for algorithms
 - Component of other data structures

Declaration of a Queue

```
# define MAXQUEUE 50 /* size of the queue items*/  
typedef struct {  
    int front;  
    int rear;  
    int items[MAXQUEUE];  
}QUEUE;
```

QUEUE Operations

- Initialize the queue
- Insert to the rear of the queue
- Remove (Delete) from the front of the queue
- Is the Queue Empty
- Is the Queue Full
- What is the size of the Queue

Initialize the QUEUE

- The queue is initialized by having the *rear* set to *-1*, and *front* set to *0*. Let us assume that maximum number of the element we have in a queue is MAXQUEUE elements as shown below.

items[MAXQUEUE-1]

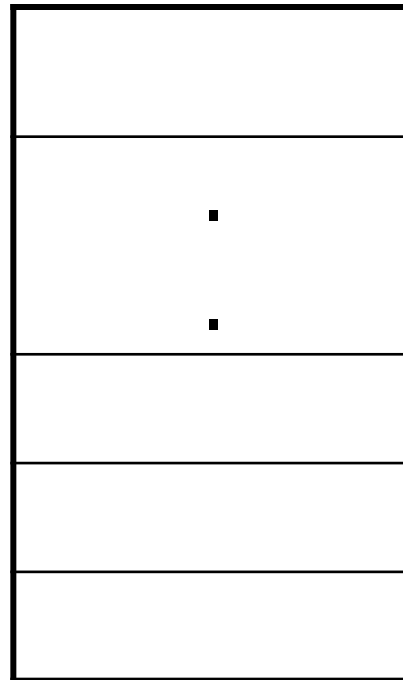
.

.

.

items[1]

items[0]

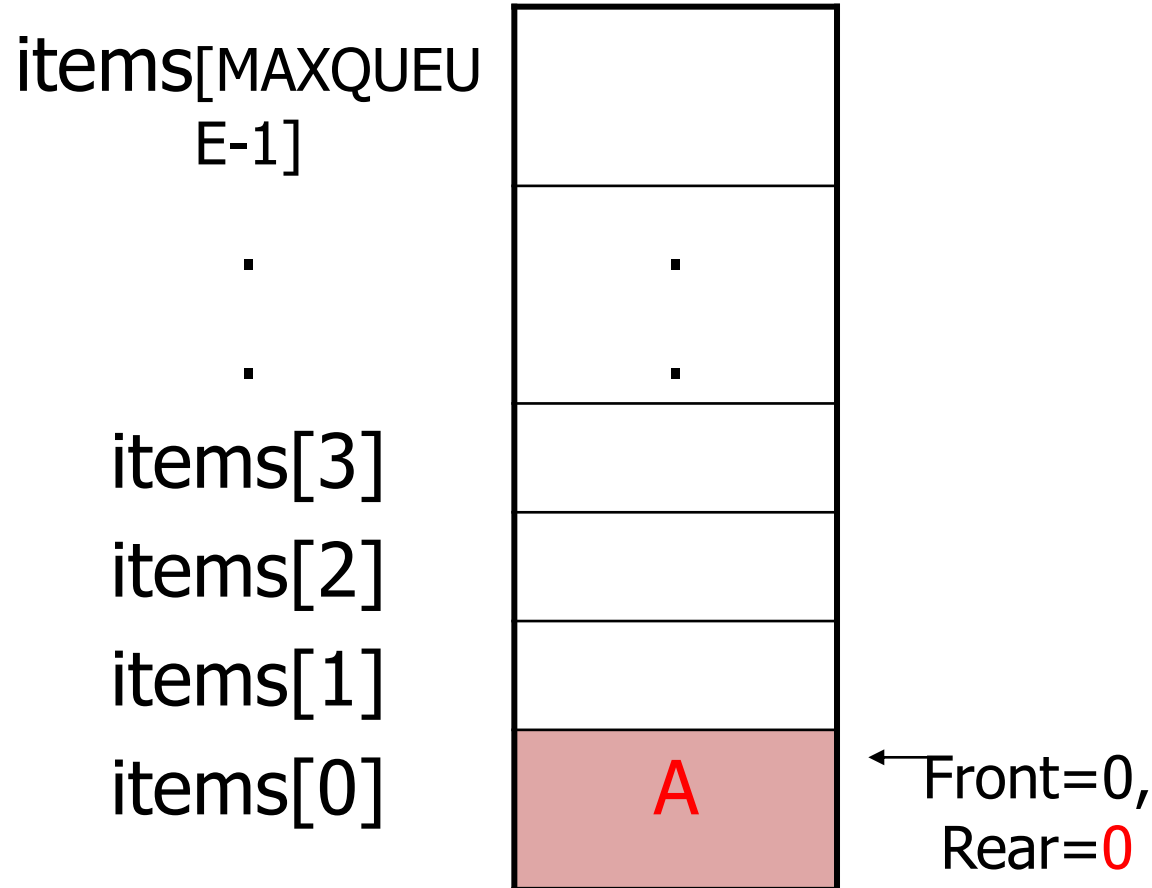


front=0

rear=-1

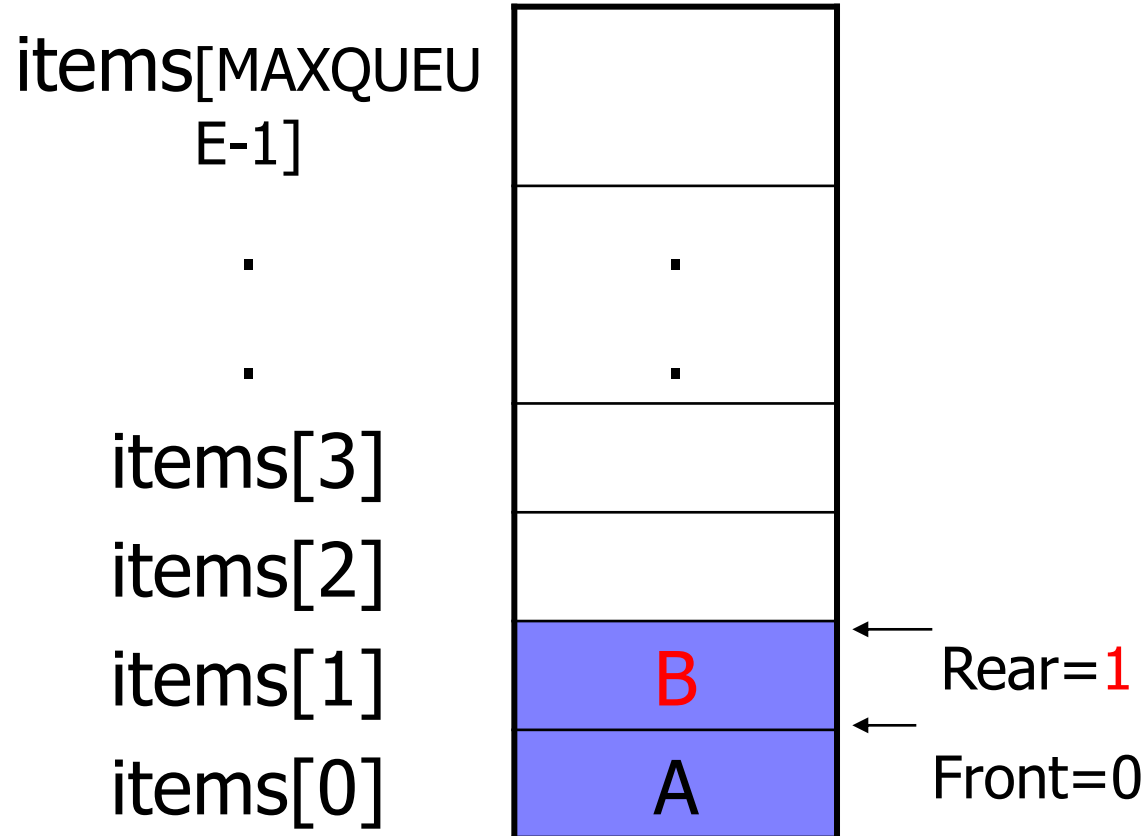
insert(&Queue, 'A')

- an item (*A*) is *inserted* at the *Rear* of the queue



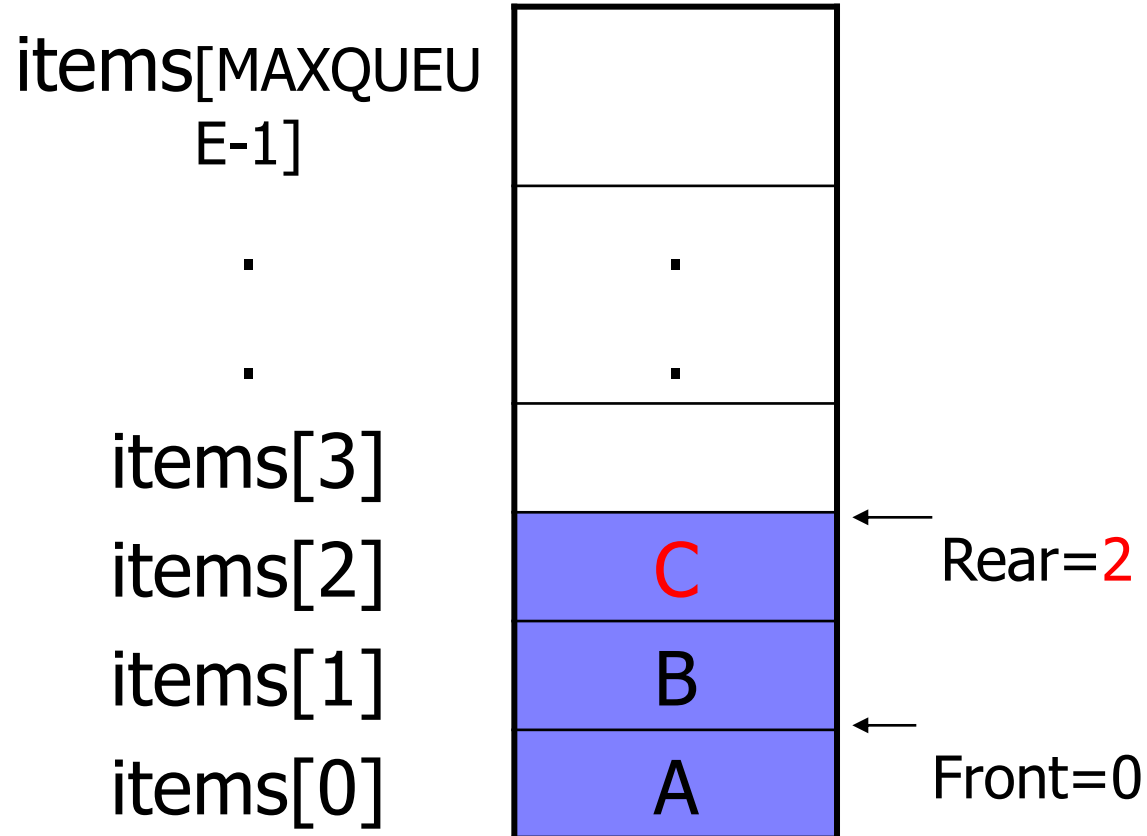
insert(&Queue, 'B')

- A new item (*B*) is *inserted* at the *Rear* of the queue



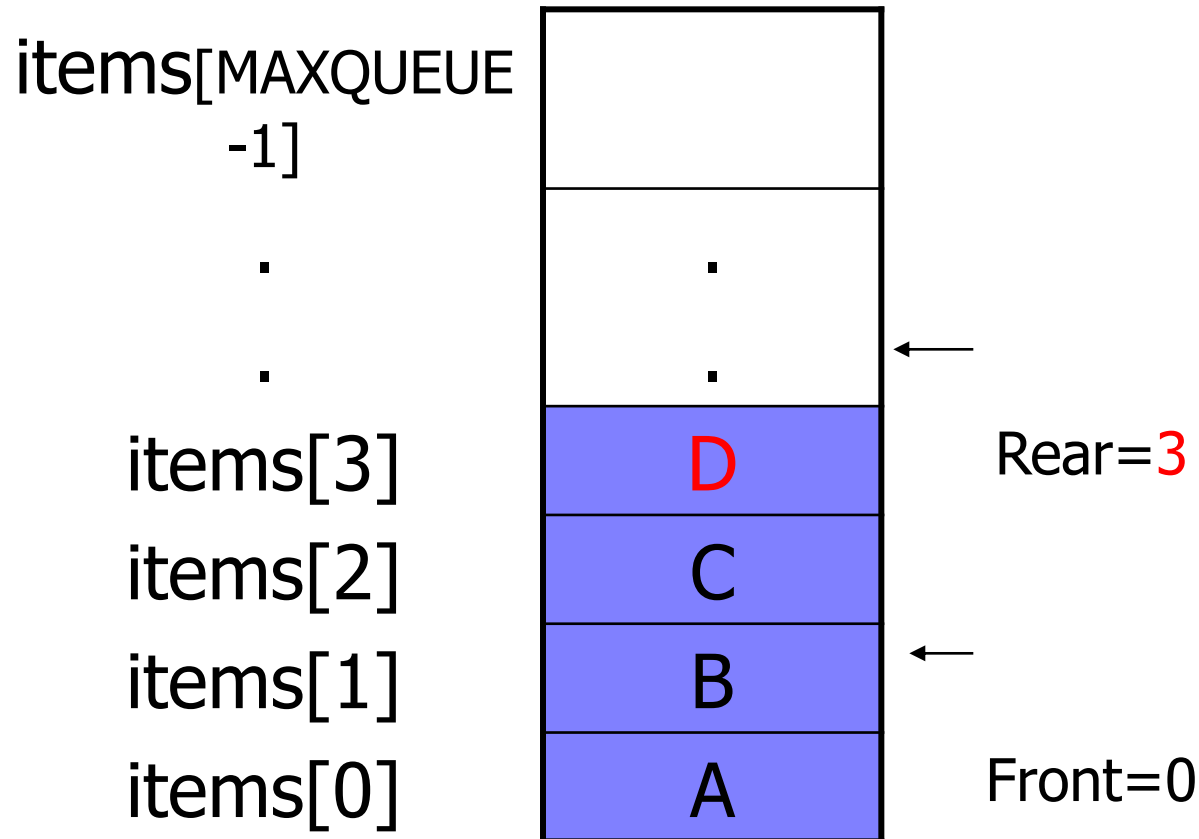
insert(&Queue, 'C')

- A new item (*C*) is *inserted* at the *Rear* of the queue



insert(&Queue, 'D')

- A new item (*D*) is *inserted* at the *Rear* of the queue

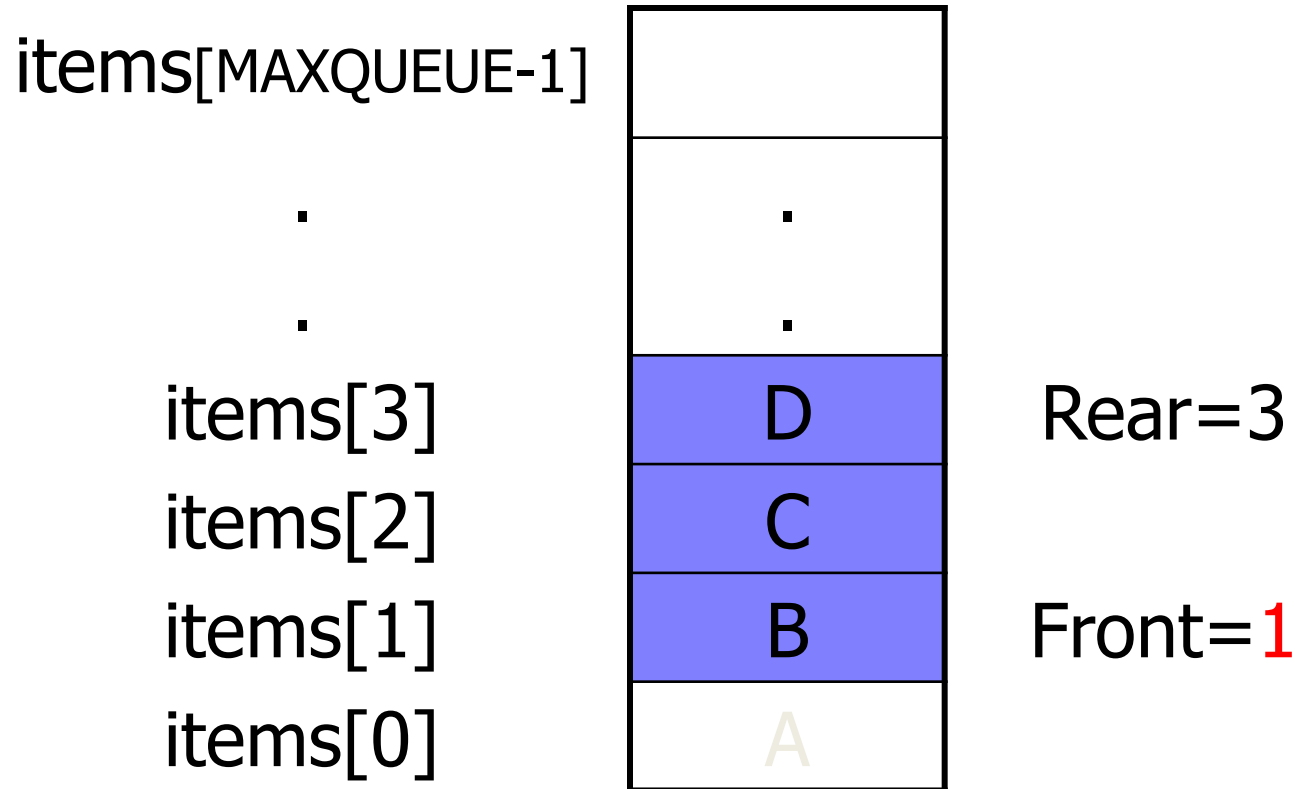


Insert Operation

```
void insert(QUEUE *qptr, char x)
{
    if(qptr->rear == MAXQUEUE-1)
    {
        printf("Queue is full!");
        exit(1);
    }
    else
    {
        qptr->rear++;
        qptr->items[qptr->rear]=x;
    }
}
```

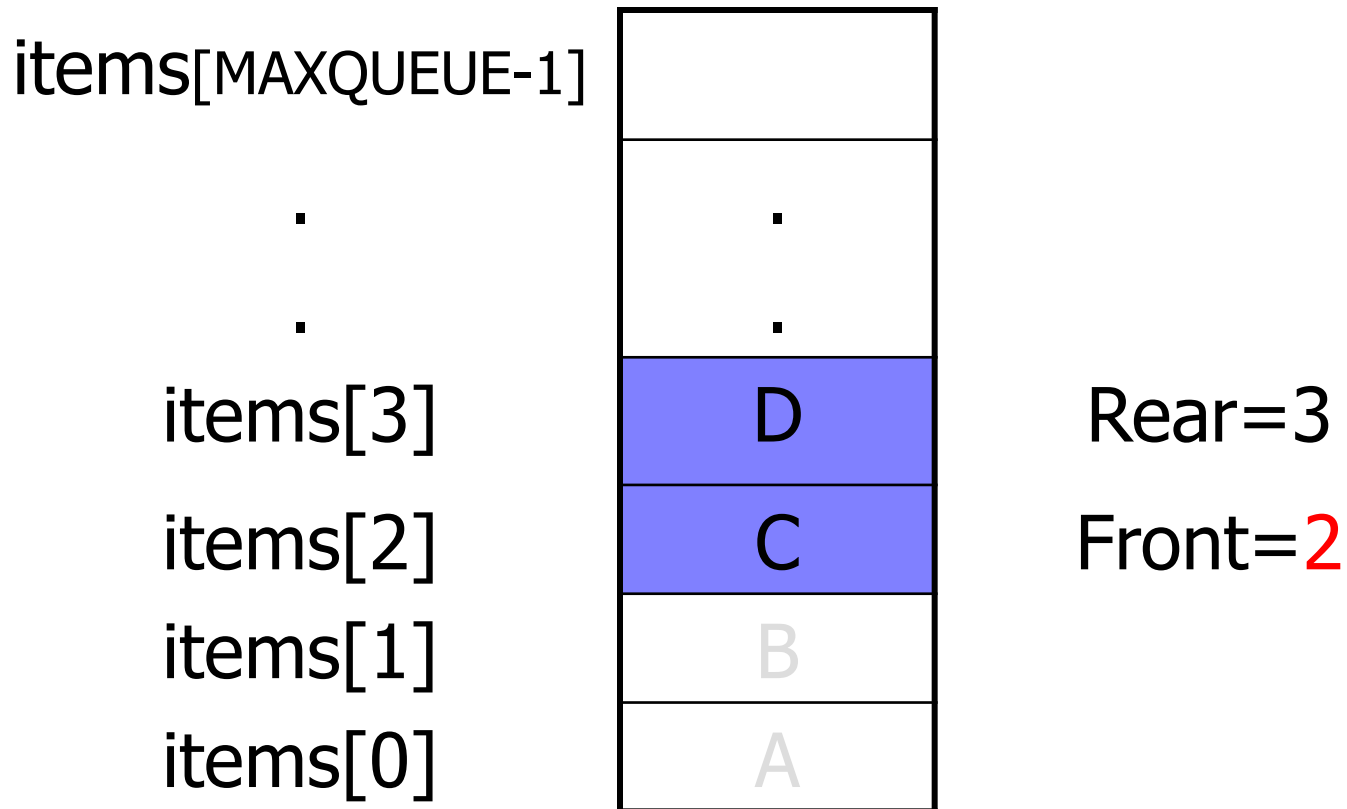
char remove(&Queue)

- an item (**A**) is removed (deleted) from the *Front* of the queue



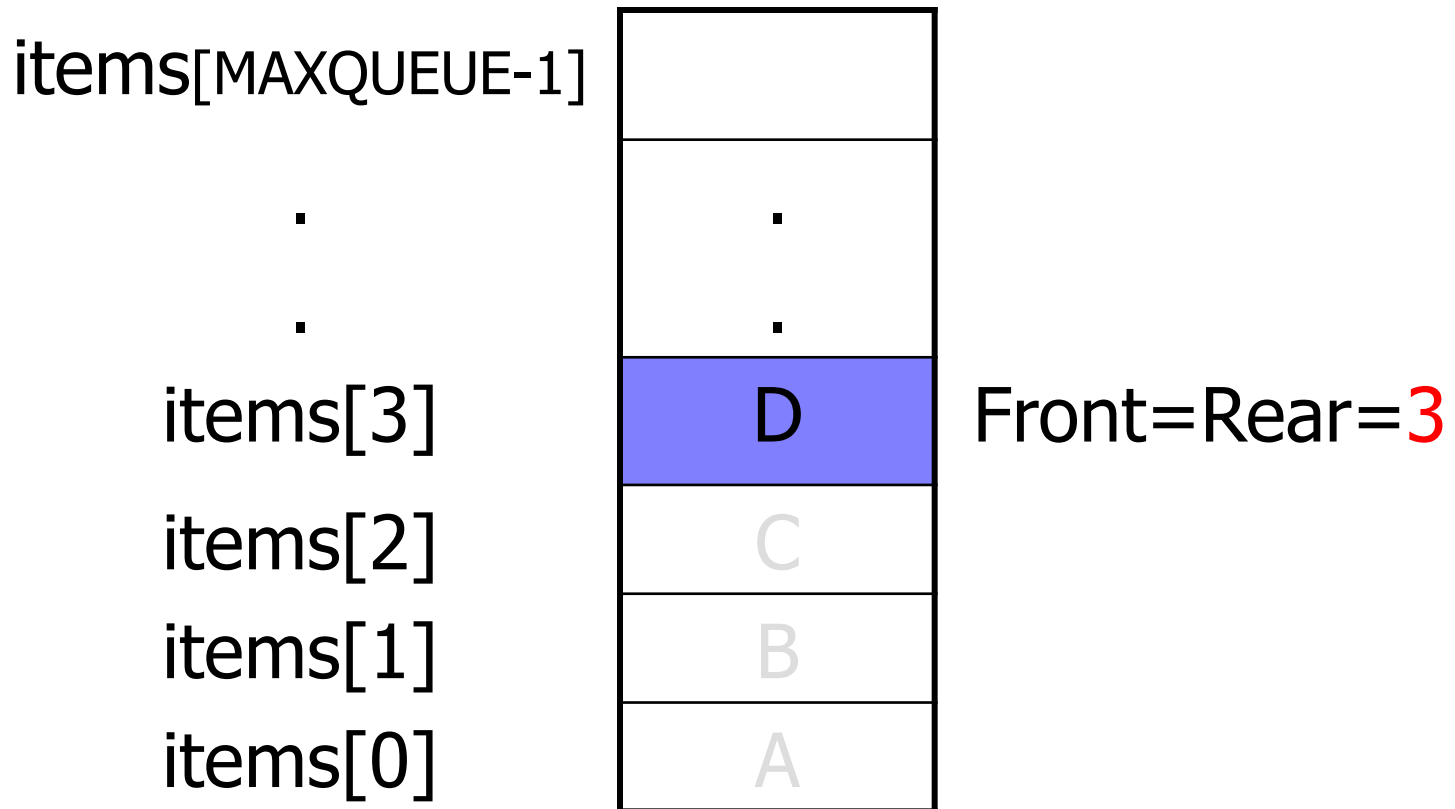
char remove(&Queue)

- ***Remove two items from the front of the queue.***



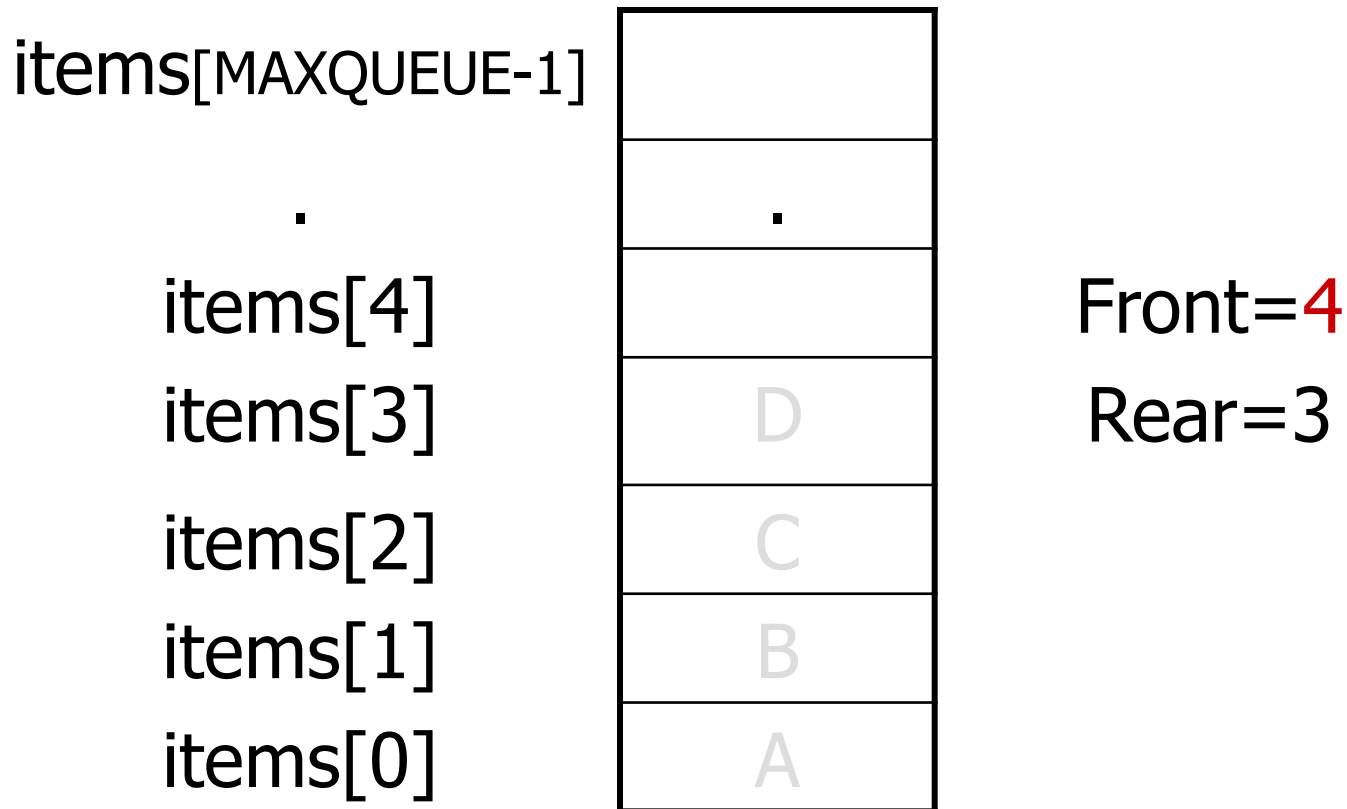
char remove(&Queue)

- ***Remove*** three items from the front of the queue.



char remove(&Queue)

- ***Remove*** one more item from the front of the queue.

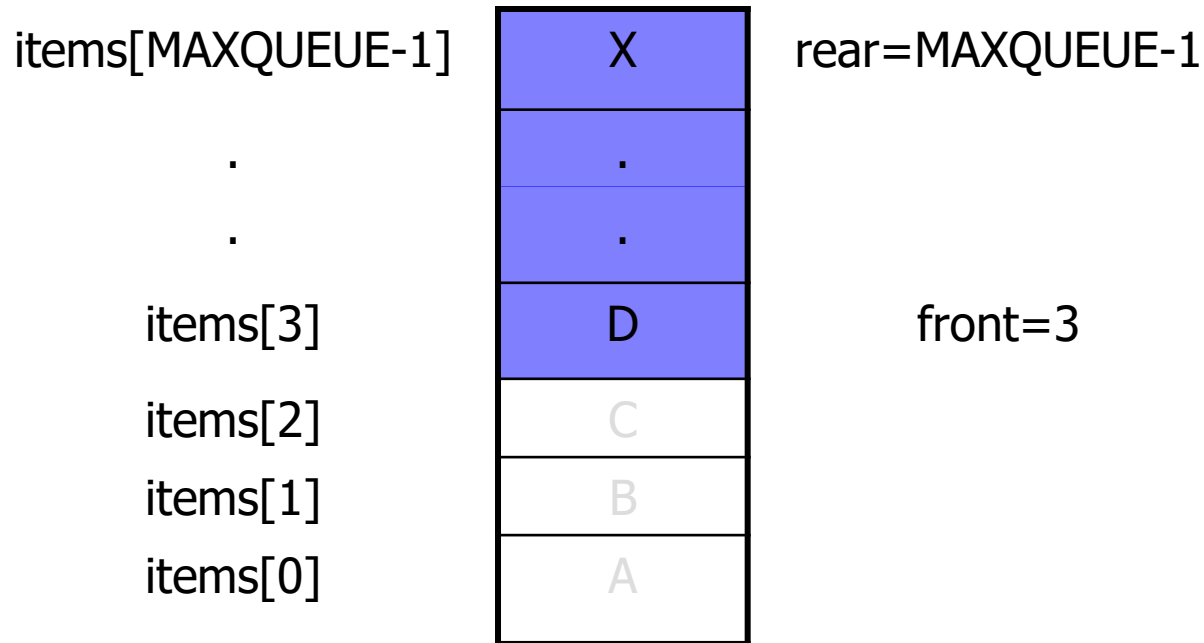


Remove Operation

```
char remove(struct queue *qptr)
{
    char p;
    if(qptr->front > qptr->rear){
        printf("Queue is empty");
        exit(1);
    }
    else
    {p=qptr->items[qptr->front];
    qptr->front++;
    return p;
    }
}
```

INSERT / REMOVE ITEMS

- *Assume that the rear= MAXQUEUE-1*

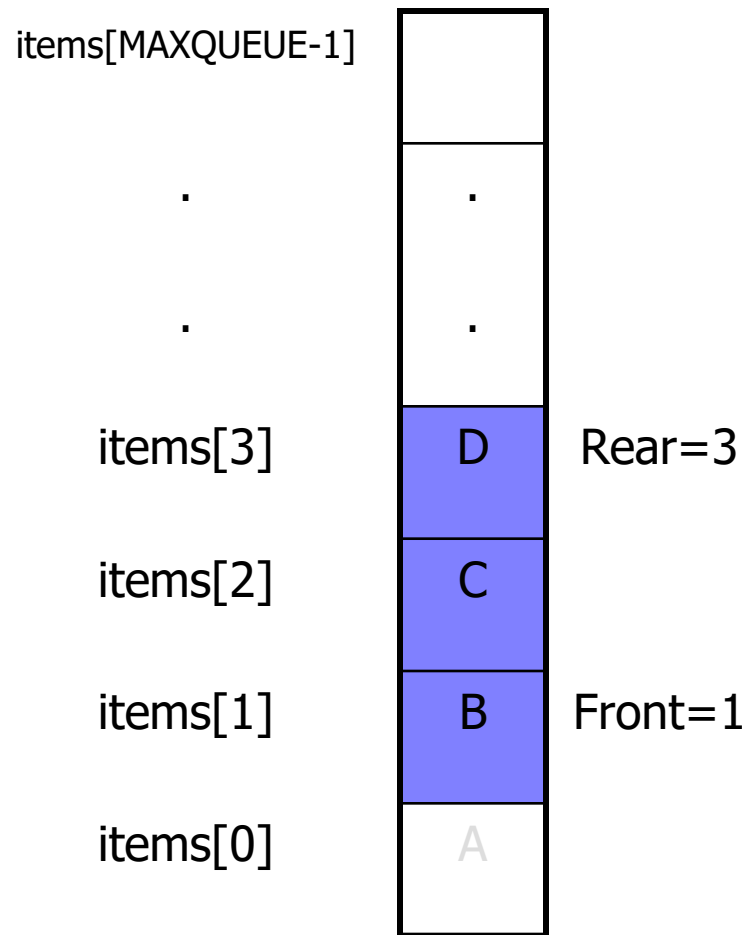


- What happens if we want to insert a new item into the queue?

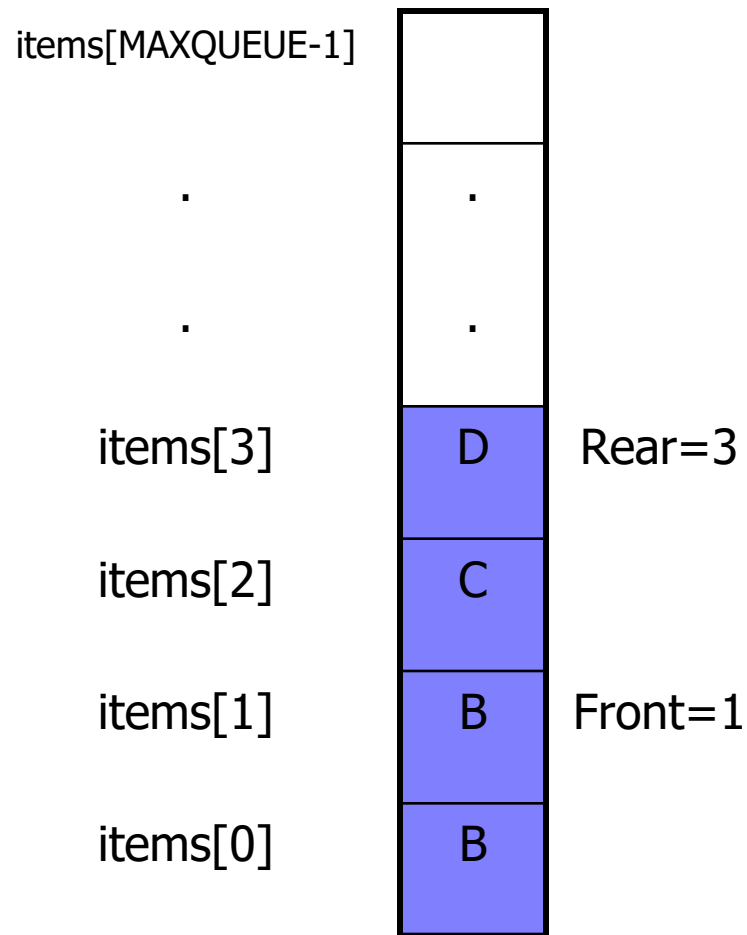
INSERT / REMOVE ITEMS

- What happens if we want to insert a new item F into the queue?
- Although there is some empty space, the queue is full.
- One of the methods to overcome this problem is to shift all the items to occupy the location of deleted item.

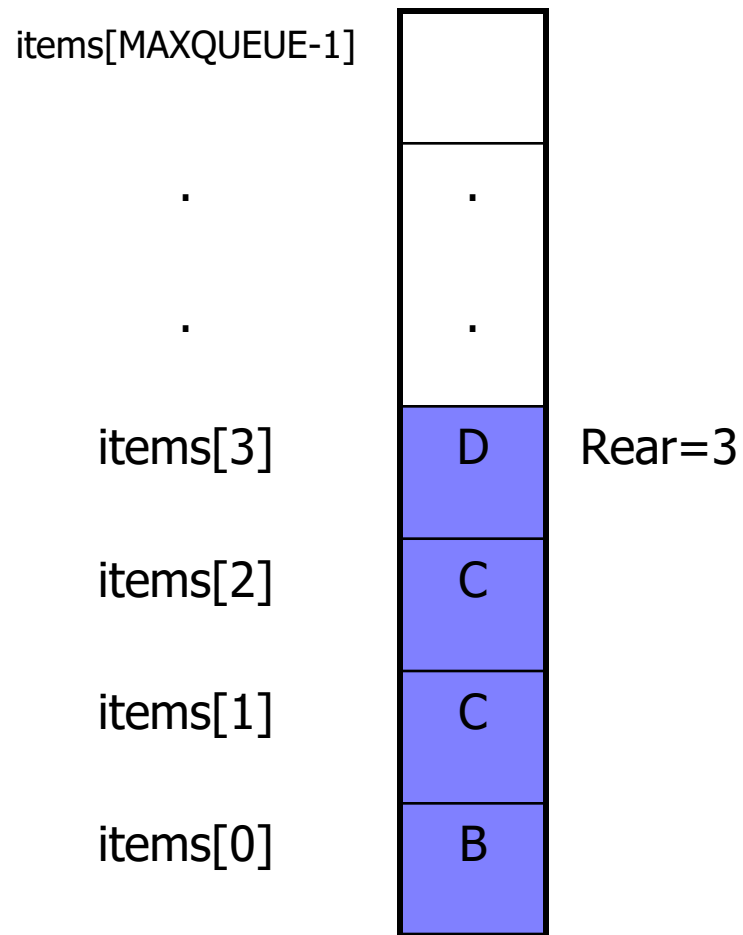
REMOVE ITEM



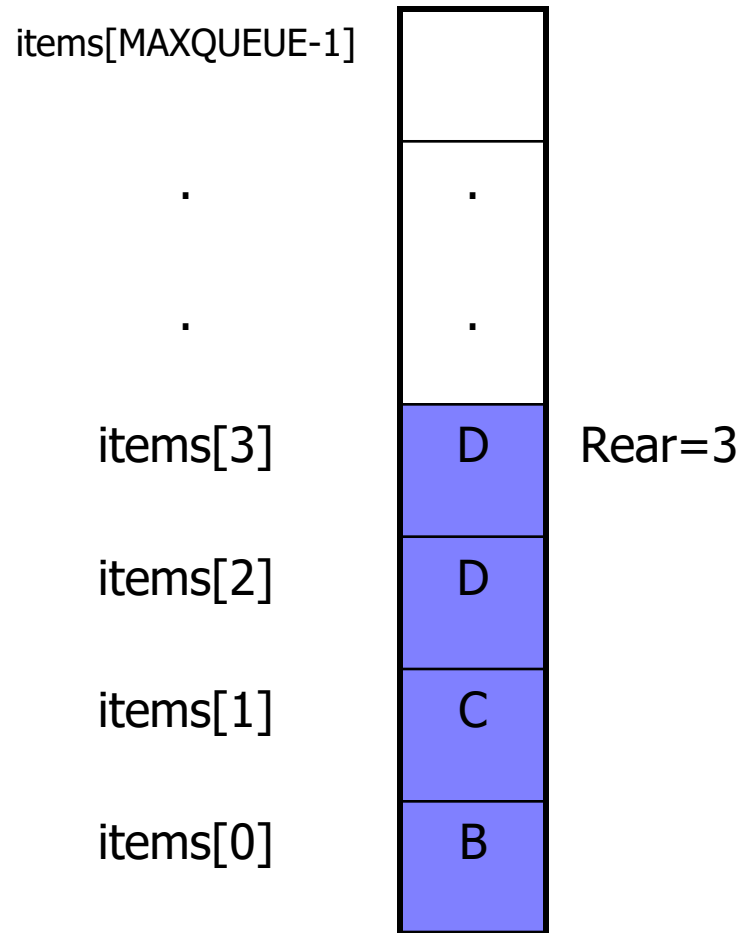
REMOVE ITEM



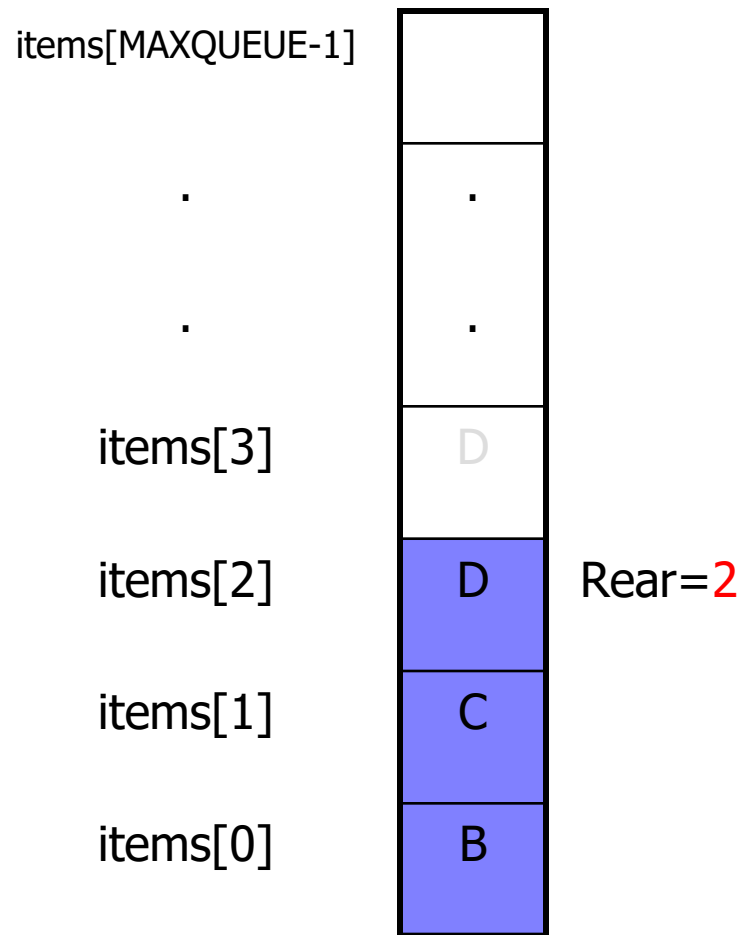
REMOVE ITEM



REMOVE ITEM



REMOVE ITEM



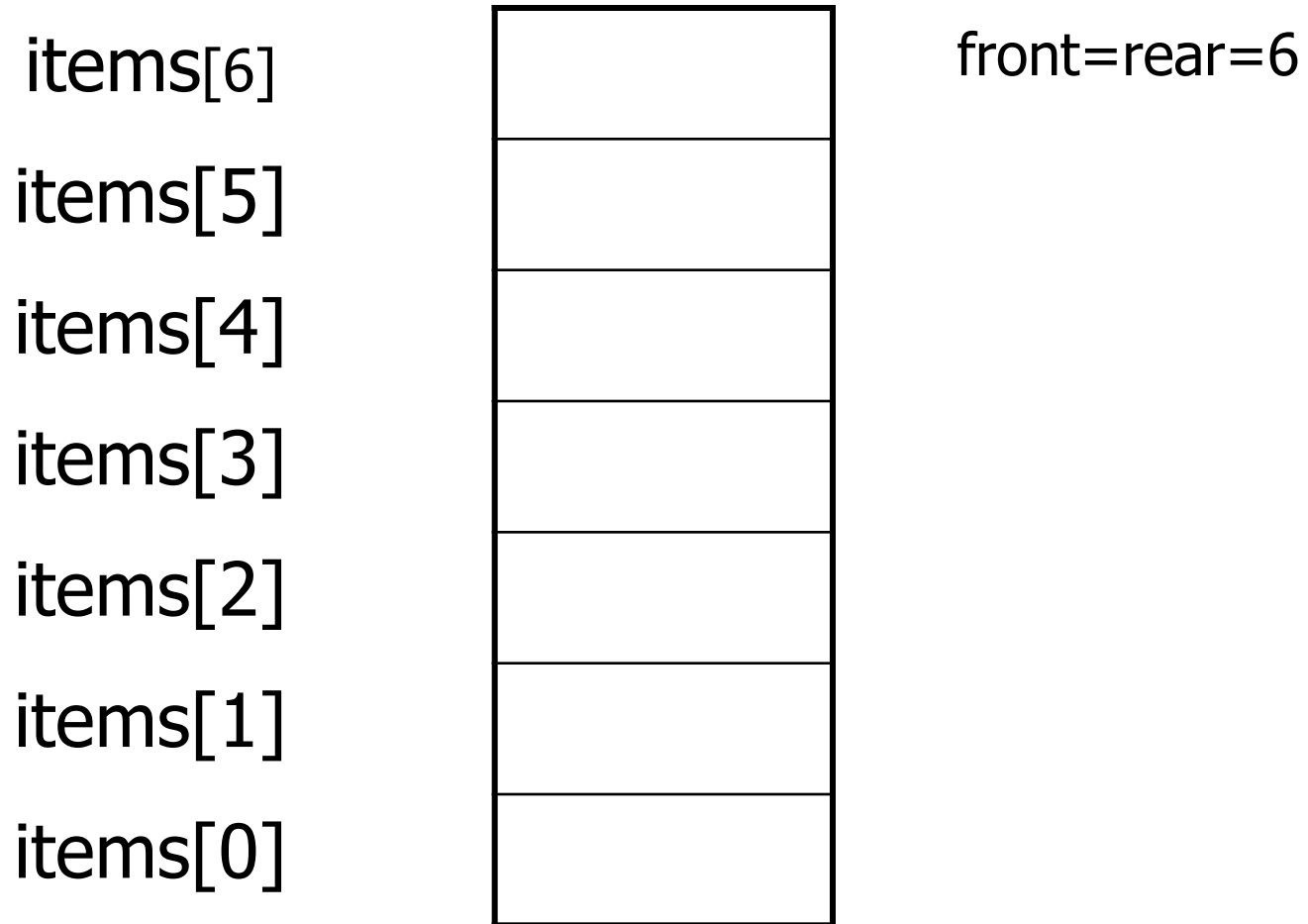
Modified *Remove* Operation

```
char remove(struct queue *qptr)
{
    char p;
    int i;
    if(qptr->front > qptr->rear){
        printf("Queue is empty");
        exit(1);
    }
    else
    {p=qptr->items[qptr->front];
    for(i=1;i<=qptr->rear;i++)
    qptr->items[i-1]=qptr->items[i];
    qptr->rear--;
    return p;
    }
}
```

INSERT / REMOVE ITEMS

- Since all the items in the queue are required to shift when an item is deleted, this method is not preferred.
- The other method is *circular queue*.
- When $\text{rear} = \text{MAXQUEUE} - 1$, the next element is entered at `items[0]` in case that spot is free.

Initialize the queue.



Insert items into circular queue

◆ ***Insert A,B,C to the rear of the queue.***

items[6]

items[5]

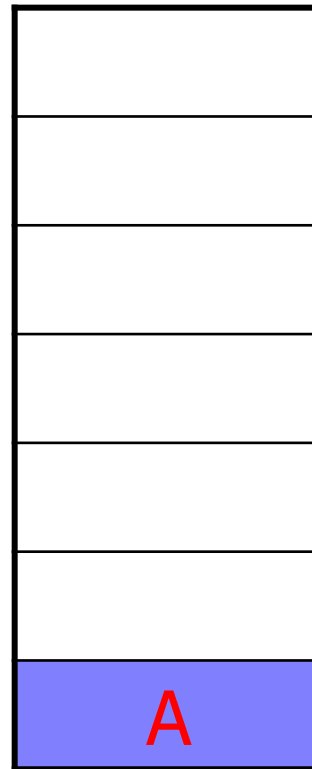
items[4]

items[3]

items[2]

items[1]

items[0]

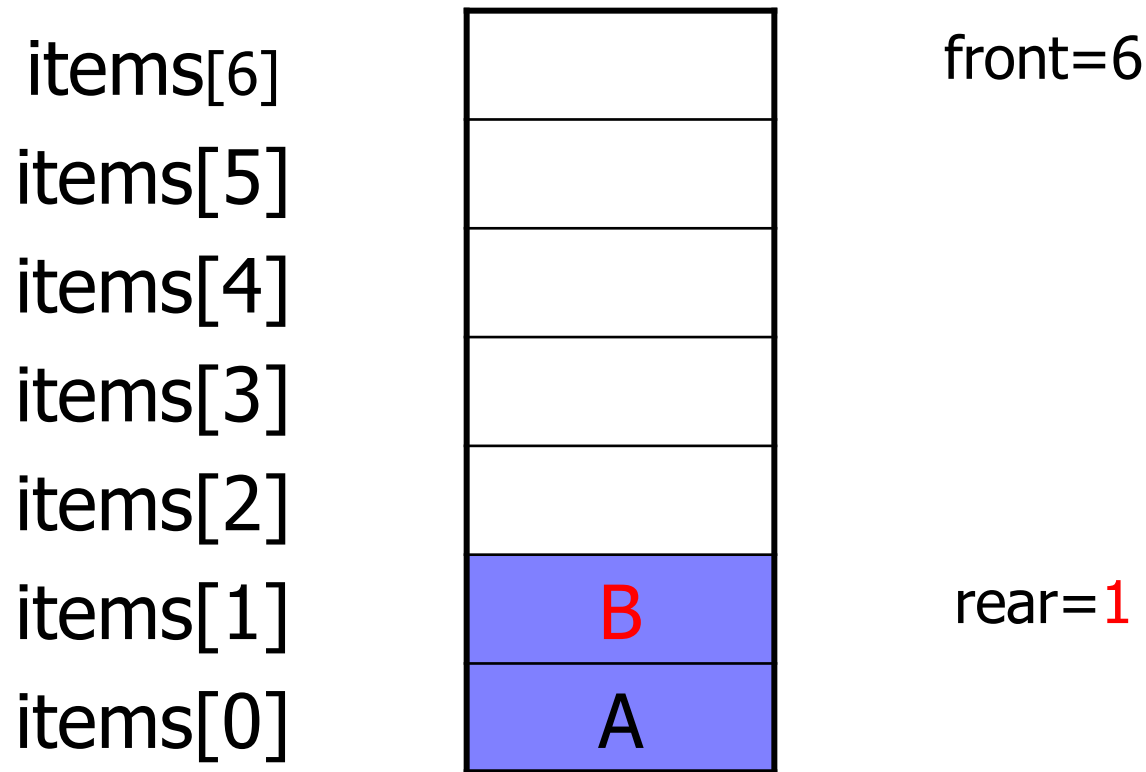


front=6

rear=0

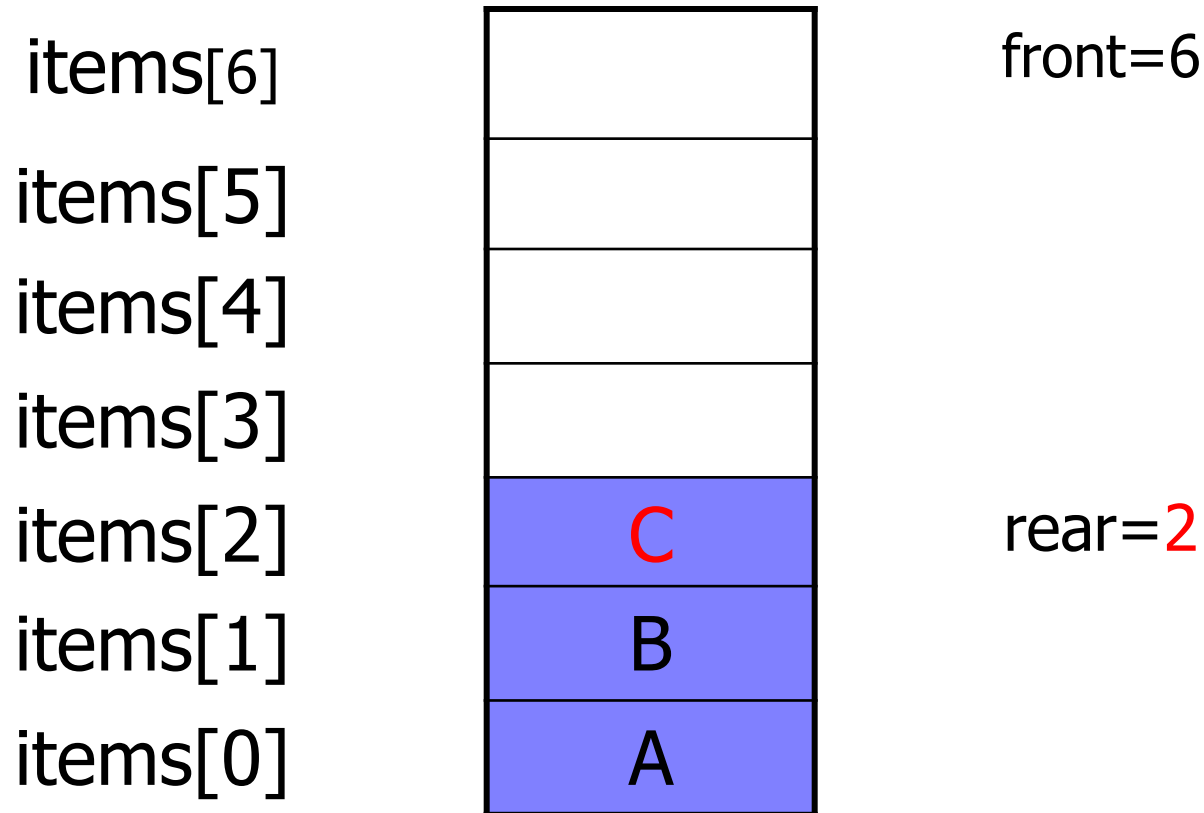
Insert items into circular queue

◆ *Insert A,B,C to the rear of the queue.*



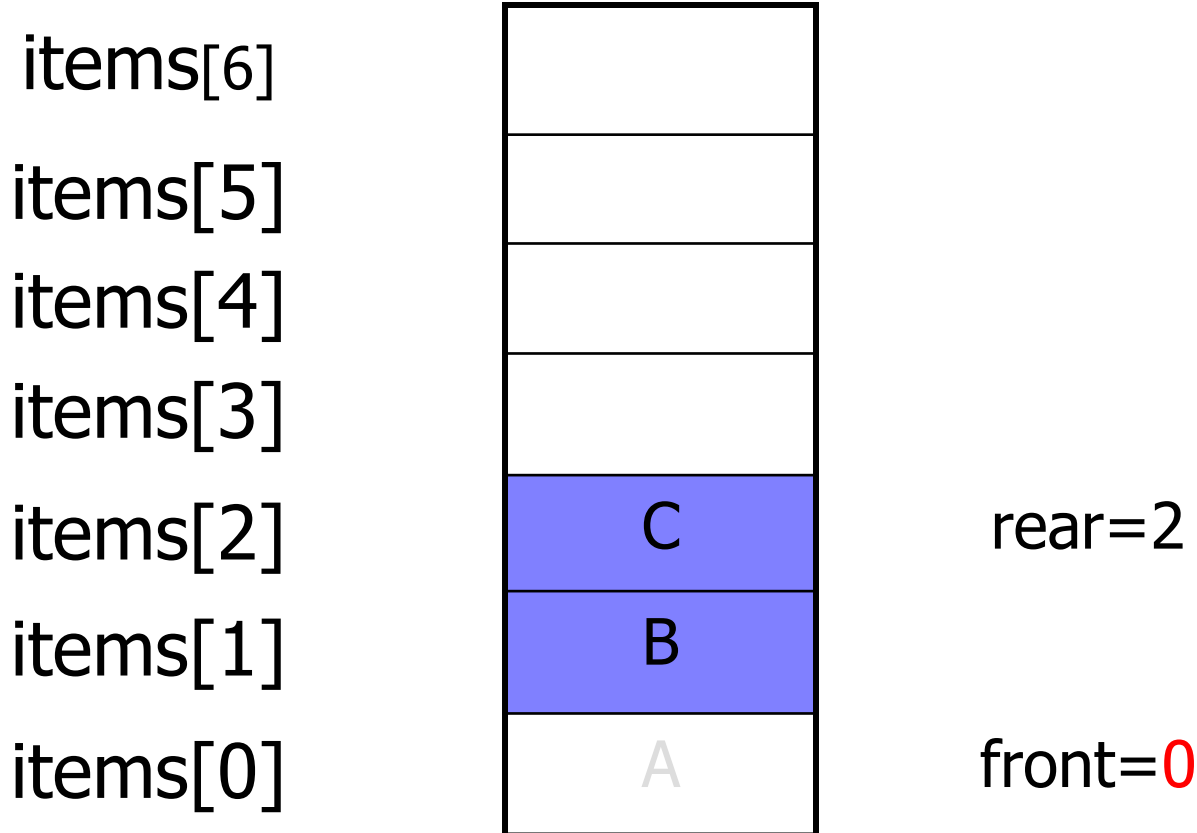
Insert items into circular queue

- *Insert A,B,C to the rear of the queue*



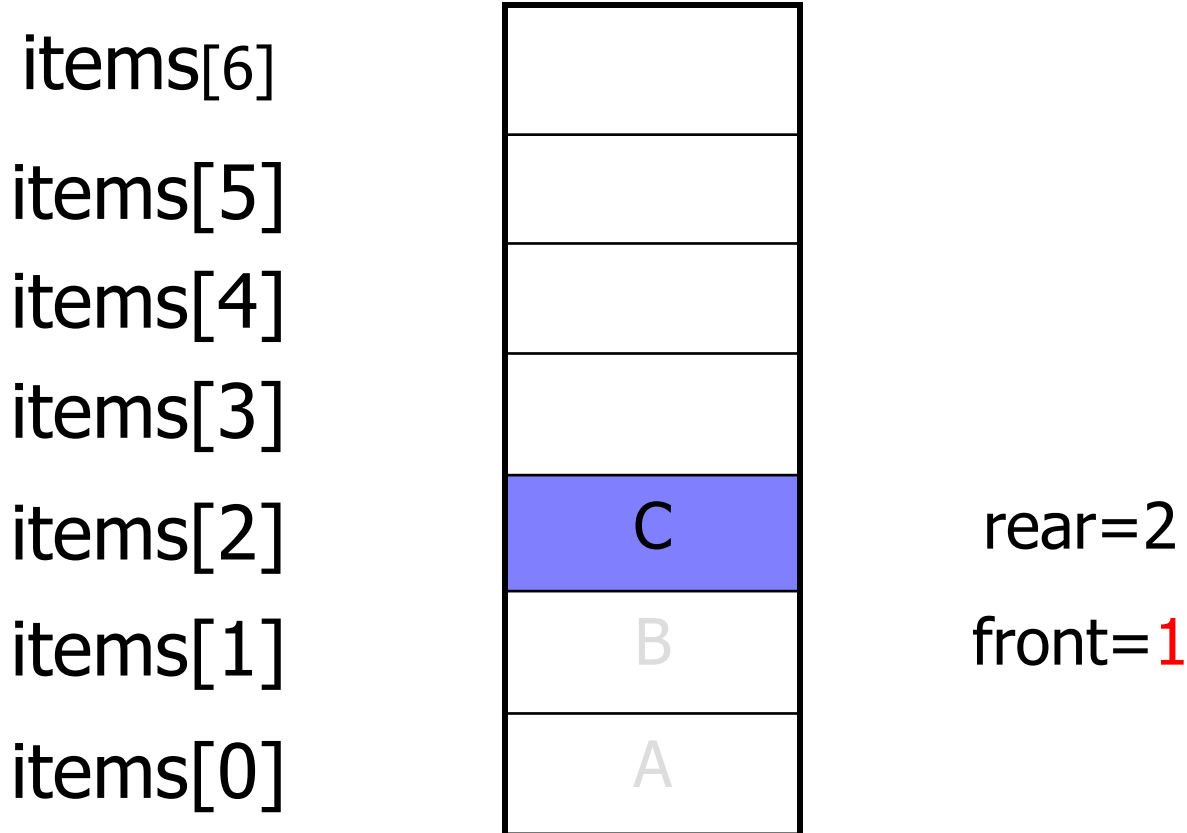
Remove items from circular queue

- ***Remove* two items from the queue.**



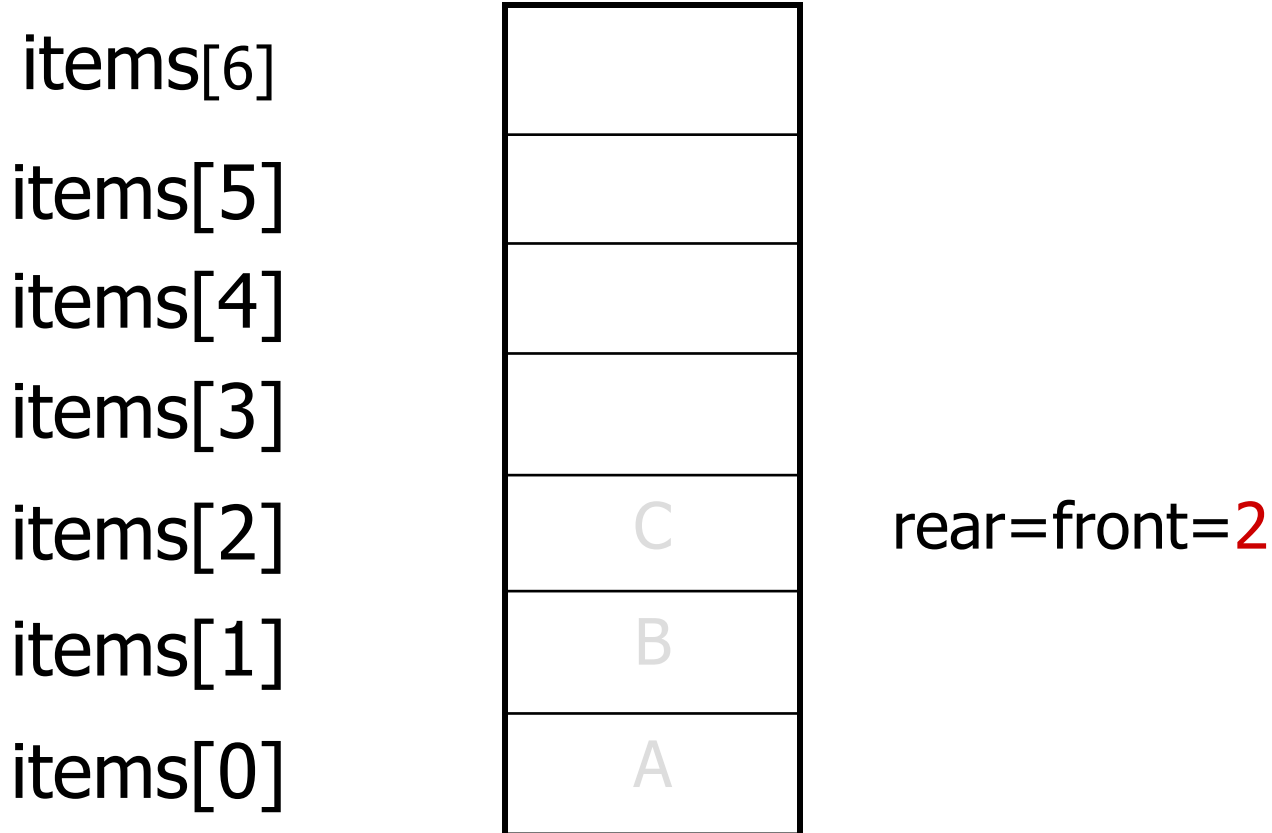
Remove items from circular queue

- ***Remove* two items from the queue.**



Remove items from circular queue

- ***Remove* one more item from the queue**



- ***Insert D,E,F,G to the queue.***

items[6]

items[5]

items[4]

items[3]

items[2]

items[1]

items[0]

G
F
E
D
C
B
A

rear=6

front=2

- ***Insert H and I to the queue.***

items[6]

items[5]

items[4]

items[3]

items[2]

items[1]

items[0]

G
F
E
D
C
B
H

front=2

rear=0

- ***Insert H and I to the queue.***

items[6]

items[5]

items[4]

items[3]

items[2]

items[1]

items[0]

G
F
E
D
C
I
H

front=2

rear=0

- ***Insert J to the queue.***

items[6]

G

items[5]

F

items[4]

E

items[3]

D

items[2]

??

front=rear=2

items[1]

I

items[0]

H

Declaration and Initialization of a Circular Queue.

```
#define MAXQUEUE 10 /* size of the queue items*/  
typedef struct {  
    int front;  
    int rear;  
    int items[MAXQUEUE];  
}QUEUE;  
QUEUE q;  
q.front = MAXQUEUE-1;  
q.rear= MAXQUEUE-1;
```

Initialization

Insert Operation for circular Queue

```
void insert(QUEUE *qptr, char x)
{
    if(qptr->rear == MAXQUEUE-1)
        qptr->rear=0;
    else
        qptr->rear++;
    /* or qptr->rear=(qptr->rear+1)%MAXQUEUE) */
    if(qptr->rear == qptr->front){
        printf("Queue overflow");
        exit(1);
    }
    qptr->items[qptr->rear]=x;
}
```


Remove Operation for circular queue

```
char remove(struct queue *qptr)
{
    if(qptr->front == qptr->rear){
        printf("Queue underflow");
        exit(1);
    }
    if(qptr->front == MAXQUEUE-1)
        qptr->front=0;
    else
        qptr->front++;
    return qptr->items[qptr->front];
}
```

PRIORITY QUEUES

- The priority queue is a data structure in which intrinsic ordering of the elements determines the results of its basic operations
- An ***ascending priority queue*** is a collection of items into which items can be inserted arbitrarily and from which only the smallest item can be removed. On the other hand a ***descending priority queue*** allows only the largest item to be removed

Priority QUEUE Operations

- **Insertion**

- The insertion in Priority queues is **the same as** in non-priority queues.

- **Deletion**

- Deletion requires a search for the element of highest priority and deletes the element with highest priority. The following methods can be used for deletion/removal from a given Priority Queue:
 - An empty indicator replaces deleted elements.
 - **After each deletion elements can be moved up in the array decrementing the rear.**
 - The array in the queue can be maintained as an ordered circular array

Priority Queue Declaration

- ***Queue data type of Priority Queue is the same as the Non-priority Queue.***

```
#define MAXQUEUE 10 /* size of the queue items*/  
typedef struct {  
    int front, rear;  
    int items[MAXQUEUE];  
}QUEUE;
```

REMOVE OPERATION

- *PriQremove* Operation using removing the element with highest priority and shifting the elements up in the array and decrementing rear. Consider Ascending Priority Queue.

```
int PRiQremove(QUEUE *qptr)
{
    int smallest, loc, f, i;
    f=qptr->front;
    if(qptr->front == qptr->rear){
        printf("Queue underflow");
        exit(1);
    }
    smallest = qptr->items[(qptr->front+1)%MAXQUEUE];
    loc = (qptr->front+1)%MAXQUEUE;
    (qptr->front++)%MAXQUEUE; /* Circular increment*/
    while(qptr->front != qptr->rear){
        if(qptr->items[(qptr->front+1)%MAXQUEUE] <smallest){
            smallest = qptr->items[(qptr->front+1)%MAXQUEUE];
            loc = (qptr->front+1)%MAXQUEUE;
        }
        qptr->front =(qptr->front+1)%MAXQUEUE; /* Circular inc.*/
    }
}
```

```
while(loc != qptr->rear){  
    qptr->items[loc] = qptr->items[(loc+1)%MAXQUEUE];  
    (loc++)%MAXQUEUE;  
}  
qptr->front=f;  
if(qptr->rear == 0) /*Decrement rear after removing one  
item*/  
qptr->rear = MAXQUEUE -1;  
else  
qptr->rear--;  
return smallest;  
}
```

Insert Operation of Priority Queue is the same as the insert of the non-priority queues.

```
void insert(struct queue *qptr, int x)
{
    qptr->rear = (qptr->rear++)%MAXQUEUE; /*Circular
    increment*/
    if(qptr->rear == qptr->front){
        printf("Queue overflow");
        exit(1);
    }
    qptr->items[qptr->rear]=x;
}
```